Gate 08 — Auth and Fetch Strategy

Status: proposed (Architecture) **Scope:** Two of four Gate 08 prerequisites — session/auth and client fetch/mutation. Builds on Gate 07 (public read surface) and the Gate 07 reconciliation Gate 08 handoff notes. **Non-scope:** story breakdown (PM), threat modeling (BA), i18n (Gate 09).

The driver for Gate 08 is Aleksei's product reframe: **authenticated CRUD on site** for Species, ConservationAssessment, ExSituPopulation, FieldProgram, SpeciesLocality — not read-only coordinator dashboards. That changes the calculus for auth ergonomics (forms, expiry mid-edit, drafts) and for the fetch/mutation layer (validation errors, optimism, revalidation). It does not reopen the Gate 07 decision to reject React Query on the read surface.

Section 1 — Session/auth strategy

Decision

DRF `TokenAuthentication` (already shipped), with the token stored in an `httpOnly`, `Secure`, `SameSite=Lax` cookie set by a Next.js route handler that proxies `/api/accounts/login/`. The browser never sees the raw token; the Next.js server attaches `Authorization: Token <key>` when calling Django.

This is a BFF-lite pattern: authenticated requests flow browser → Next.js server → Django. The auth cookie is same-site with the Next.js origin, so the cross-origin seam (Vercel ↔ `api.malagasyfishes.org`) stays credential-free.

Rationale

- **Cross-origin.** A Django session cookie would need `SameSite=None; Secure` plus `CORS_ALLOW_CREDENTIALS=True` plus tightening `CORS_ALLOWED_ORIGIN_REGEXES` to a non-regex allow-list — which breaks `*.vercel.app` preview deploys. Keeping the cookie same-site with Next.js sidesteps the whole thing; Django-side CORS stays `ALLOW_CREDENTIALS=False`.
- **Field-biologist mobile context.** Spotty 3G, backgrounded PWAs. A long-lived token + `httpOnly` cookie survives restarts cleanly; JWT refresh-token rotation does not.
- **Revocation matters.** Deprovisioning a former coordinator must take minutes, not TTL. DRF's token row delete is O(1); JWT blacklists reintroduce a server-side store anyway, eliminating JWT's stateless argument.
- **Minimum migration.** Token auth is already in `REST_FRAMEWORK.DEFAULT_AUTHENTICATION_CLASSES`; `/api/accounts/login/` already returns `{token, access_tier, user_id}`; `/api/accounts/logout/` already deletes the row. We wrap, we don't rewrite.

Rejected alternatives

- **Django `SessionAuthentication` cross-origin.** Forces `CORS_ALLOW_CREDENTIALS=True` and a non-regex origin allow-list, breaks Vercel previews, adds CSRF surface to every mutation. More config, more surface, no user-facing benefit.
- **JWT (SimpleJWT).** Gives us nothing we need at one Django node. Refresh rotation adds 401-retry races on form submissions; deprovisioning still needs a server-side blacklist. Revisit only if a native mobile client lands post-MVP.
- **Raw token in `localStorage`.** XSS-exfiltratable; called out as unacceptable.
- **NextAuth / Auth.js.** Would either duplicate the Django user table or reduce to a thin wrapper around our login endpoint. We'll just write the thin wrapper.

Integration sketch

Django — minimal additions on top of what's shipped:

- Existing: `POST /api/accounts/login/`, `POST /api/accounts/logout/`, `GET /api/accounts/me/`.
- **New:** token TTL. DRF tokens are forever by default. Add `DRF_TOKEN_TTL_HOURS=168` (7d, sliding) via a small `TokenAuthentication` subclass that 401s and deletes when stale, and bumps a `last_used_at` column on each successful auth.
- **New, deprovisioning:** signal on `User.is_active=False` or `access_tier` downgrade → delete that user's `Token` rows + write `audit.AuditLog` entry.

Next.js — new server-only module `frontend/lib/auth.ts`:

- `POST /app/api/auth/login/route.ts` — forwards to Django, on success writes cookie `mffcp_session` (`httpOnly; Secure; SameSite=Lax; Path=/; Max-Age=604800`). Returns `{access_tier}` for tier-aware UI rendering only (authz is enforced server-side).
- `POST /app/api/auth/logout/route.ts` — forwards to Django, unsets the cookie.
- `lib/auth.ts#getSessionToken()` — server-only, reads `cookies()` from `next/headers`; guard with `"server-only"`.
- `lib/auth.ts#getCurrentUser()` — hits `/api/accounts/me/`, cached 60s via `unstable_cache` keyed by SHA-256 of the token.
- **Middleware** (`frontend/middleware.ts`) — `/edit/*` and `/coordinator/*` without cookie → 302 `/login`.

`lib/api.ts` change. The header-injection hook flagged in Gate 07 reconciliation §4.2 was not implemented. Adding it:

```
// sketch — additions only
import { cookies } from "next/headers"; // server-only

async function buildAuthHeaders(): Promise<HeadersInit> {
  const token = cookies().get("mffcp_session")?.value;
  return token ? { Authorization: `Token ${token}` } : {};
}

export async function apiFetch<T>(path: string, options: ApiFetchOptions = {}):
Promise<T> {
  const auth = options.auth === false ? {} : await buildAuthHeaders();
  const res = await fetch(`${resolveBaseUrl()}${path}`, {
    method: options.method,
    body: options.body,
    headers: { Accept: "application/json", ...auth, ...options.headers },
    signal: options.signal,
    // Invariant: authenticated fetches never participate in ISR.
    ...(options.auth === false
      ? { next: { revalidate: options.revalidate ?? DEFAULT_REVALIDATE } }
      : { cache: "no-store" }),
  });
  if (!res.ok) throw new ApiError(res.status, path, await res.text());
```

```
  return res.status === 204 ? (undefined as T) : ((await res.json()) as T);
}
```

The invariant — `cache: "no-store"` on any call carrying `Authorization` — prevents poisoning the public ISR cache with coordinator data.

CORS / CSRF

- **CORS stays as-is.** `CORS_ALLOW_CREDENTIALS=False`; the `*.vercel.app` regex keeps working.
- **CSRF.** Not applicable to token-auth on Django. The cookie-setting endpoint is the Next.js login route — `SameSite=Lax` blocks classic form-submit CSRF; add an `Origin` header check in the route as defense in depth.
- **Admin** keeps same-origin session auth at `api.malagasyfishes.org/admin/`. Unchanged.

Logout + expiry mid-form

Coordinator is mid-edit on a breeding recommendation when her token expires.

1. **Early detection.** Layout around `/coordinator/*` calls `getCurrentUser`; at T-10 min, a banner "Session expires in 10 min — Extend" calls a `/api/auth/refresh` route that bumps `last_used_at`.
2. **In-flight work.** The form auto-persists to `sessionStorage` on every change (`/edit/species/<id>/draft`). On a 401 at submit, the Server Action returns `{ kind: "auth_expired" }`; the client shows a modal "Session expired — log in to save" with a "Keep editing locally" escape. After re-login the form re-hydrates from `sessionStorage`.
3. **Hard expiry.** 401 from Django → Next.js clears the cookie → no silent retry.

Deprovisioning

- `User.is_active=False` or tier downgrade → signal deletes `Token` rows + writes audit entry.
- The former coordinator's next request: Django 401 → Next.js clears cookie → `/login`.
- **Max revocation window: 60s** (the `getCurrentUser` cache TTL). Admin can force-flush via the existing `/api/revalidate` webhook extended with a `user:<id>` cache tag.

Section 2 — Fetch / mutation strategy

Decision

Reads: unchanged from Gate 07 — `apiFetch` from Server Components, ISR for public, `no-store` for authenticated. Writes: Next.js Server Actions. No React Query, no SWR. Client-side optimism via React 19 `useOptimistic` + `useFormState`.

Rationale

- **What React Query would buy us:** (a) mutation state, (b) automatic invalidation, (c) optimistic updates. React 19 covers (c) natively. `revalidatePath` / `revalidateTag` in Server Actions covers (b) with a shorter mental model than query keys. (a) is `useFormState` + `pending`. Net: the ergonomics without a duplicate cache layer.
- **Gate 07 rejected React Query for reads.** Writes don't change the math — editing is form-heavy, not list-heavy, and post-save lists are small enough that a full RSC re-fetch after `revalidatePath` is trivial.
- **Auth alignment.** Server Actions run on the Next.js server, read the cookie, call `apiFetch` with `Authorization` injected. Token never leaves the server. React Query would push client-side

fetches, which means token-in-browser or every-call-proxied — at which point Server Actions are strictly simpler.
- **DRF validation errors map cleanly.** DRF's `{field: ["error"]}` is exactly `useFormState`'s shape.

Interaction with the auto-revalidate webhook

1. Coordinator saves. Server Action calls Django via `apiFetch`.
2. Django commits row + audit row in the **same transaction** (`transaction.on_commit` triggers the signal).
3. `post_save` signal, via `on_commit`, POSTs to `/api/revalidate` with affected paths. **Fire-and-forget**; failures log-and-warn, never 500 the user's save.
4. The Server Action **also** calls `revalidatePath(...)` locally before returning — belt-and-braces so the editor never sees their own stale write.
5. RSC tree re-renders with fresh data.

Latency budget: in-Action `revalidatePath` is synchronous with the response, zero cost. The Django→webhook leg is off the critical path; if it fails, the next ISR window (1h) catches it and the editor is already correct via step 4.

Rejected alternatives

- **React Query / SWR.** ~40kb plus a second cache and parallel invalidation model. No benefit for our form-shaped workload.
- **Client `fetch` + manual state.** Leaks token or requires a proxy per endpoint. Server Actions are the proxy, cleaner.
- **tRPC.** Would require sharing types across Python/TS or regenerating from OpenAPI. drf-spectacular → TS client is a post-MVP orthogonal decision.

Representative flow — coordinator updates a species's common name

```
1. /edit/species/<id>/  (RSC)
   apiFetch(`/api/v1/species/<id>/`, { auth: true })  // no-store
   renders <EditSpeciesForm initial={...} />         // client component

2. User edits and submits; <form action={updateSpeciesAction}>.
   useOptimistic swaps the displayed value immediately.

3. updateSpeciesAction ("use server"):
   const res = await apiFetch(`/api/v1/species/${id}/`, {
     method: "PATCH", auth: true,
     headers: { "Content-Type": "application/json" },
     body: JSON.stringify({ common_name }),
   });
   // 400 → return { errors: res.body }  (useFormState renders field errors)
   // 401 → return { kind: "auth_expired" }
   // 200:
   revalidatePath(`/species/${slug}`);
   revalidatePath(`/species/`);
   revalidatePath(`/edit/species/${id}`);
   return { ok: true };

4. Django SpeciesViewSet.partial_update:
```

```
   - serializer.save() under transaction.atomic()
   - audit.log(user, "species.update", diff)   // same txn
   - transaction.on_commit(lambda: fire_revalidate_signal(species))

5. Signal handler POSTs /api/revalidate with affected paths; logs on failure.

6. Next reader sees new common name; the editor saw it via step 3.
```

Optimism in step 2; truth in step 3's `revalidatePath`. On error, `useOptimistic` unwinds and field errors render.

Error handling

- **400 (validation)** → Server Action returns `{ errors: {...} }`; per-field rendering; no toast.
- **401 (expired)** → `auth_expired`; client re-login modal preserving `sessionStorage` draft.
- **403 (tier)** → inline banner; logged as UI/authz drift (shouldn't happen if UI is tier-aware).
- **Network / 5xx** → user-visible Retry button. **No automatic retry** on mutations — non-idempotent PATCH risk.

Testing strategy

- **Server Action unit tests** (Vitest): mock `apiFetch`, assert action shape per status. No browser.
- **Django integration tests** (`TransactionTestCase` so `on_commit` actually fires): PATCH → audit row exists → webhook stub received expected paths.
- **Seam test**: Django test POSTs to a running Next.js `/api/revalidate` with a mock secret; assert 200. Covers the wire.
- **Optimistic UI**: React Testing Library with a controlled-promise fake action. Deterministic, no network.
- **Out of scope for Gate 08 CI:** Playwright E2E against live Django. The seam tests cover integration risk; full E2E is Gate 09 hardening.

Section 3 — Interactions the PM needs to know

Story ordering

- **BE-08-A (auth hardening: token TTL, `last_used_at`, deprovisioning signal, `/me` shape freeze) must ship before any FE-08 form story.** Otherwise FE is guessing at the session contract.
- **FE-08-A (Next.js auth proxy, `lib/auth.ts`, middleware, `apiFetch` auth-header hook) must ship before any FE-08 editing story.** The `apiFetch` change is part of FE-08-A, not per-form.
- **BE-08-C (Django signals → webhook) can land in parallel** with editing stories — each Server Action includes its own `revalidatePath` so editor-visible freshness is independent of BE-08-C.

BE-08-C constraints

- **Audit + mutation in one transaction.** Webhook must fire from `transaction.on_commit`, not from the ViewSet directly. Otherwise a crash between them busts the public cache with no audit trail.
- **Webhook is fire-and-forget.** A failing webhook must not fail the user's save. Log + metric; HTTP response unchanged.
- **Bulk edits must batch.** The weekly IUCN sync touches many species; firing N webhooks in a loop will hammer our own Vercel route. AC: "a task updating >5 species debounces to one webhook call with the full path list."
- **Keep the `paths: string[]` payload permissive** (as in the existing handler). Don't narrow.

Gotchas the PM should bake into ACs

1. **No ISR on authenticated routes.** `/edit/*`, `/coordinator/*` must not carry ISR headers; `apiFetch` without `{ auth: false }` must `no-store`.
2. **`sessionStorage` draft recovery is first-class.** AC: "closing the tab mid-edit and reopening within 24h restores the draft; successful submit clears it."
3. **Tier-aware UI is rendered, not enforced.** Every edit endpoint must have a server-side 403 test independent of the UI hiding the button.
4. **Preview deploys log in independently.** Cookie is scoped to the Next.js origin, so `*.vercel.app` previews work without Django CORS changes.
5. **Logout clears both ends.** AC: "logout deletes the DRF Token row AND unsets the cookie; a second request with the old cookie 401s, not a cached `/me`."
6. **Deprovisioning RTO is 60s.** AC: "after `is_active=False`, next protected request within 60s may succeed; within 120s must 401."
7. **Optimistic unwind on 400.** AC: "a 400 response restores the pre-edit display and renders field errors; no flash of optimistic state persists after error."

Open questions for Aleksei

1. **Token TTL.** 7d sliding is my default. 24h means more re-logins on patchy connections; 30d widens deprovisioning window. Pick before BE-08-A starts.
2. **2FA for Tier 4/5.** Not scoped here. If yes, it's a separate BE-08 story and pushes FE-08 login UI ~1 week.
3. **Server-side draft auto-save.** Section 1 uses client `sessionStorage`. A `POST /api/drafts/` would let someone start on a phone and finish on a laptop. Recommendation: defer to Gate 09.